

Assignment-20.5

Name: T. VIKAS

Batch: 43

H. No: 2303A52281

Task 1 – Password Storage Vulnerability Check

Analyze an AI-generated signup script for insecure password storage.

Prompt:

Generate a Python user registration and login program that stores user credentials. Then improve it using secure password hashing instead of plain text storage.

Code:

```
assgn20.5 task1.py > register_user
1 #Analyze an AI-generated signup script for insecure password storage.
2 #Prompt AI to generate a Python user registration program.
3 #Check whether passwords are stored in plain text.
4 #Improve the script using hashing (bcrypt or hashlib).
5 import bcrypt
6 # Function to register user securely
7 def register_user(username, password):
8     # Generate salt and hash password
9     hashed_password = bcrypt.hashpw(password.encode('utf-8'), bcrypt.gensalt())
10
11     # Store in file
12     file = open("users_secure.txt", "a")
13     file.write(username + "," + hashed_password.decode('utf-8') + "\n")
14     file.close()
15
16     print("User registered successfully!")
17 # Function to verify login
18 def login_user(username, password):
19     file = open("users_secure.txt", "r")
20
21     for line in file:
22         stored_username, stored_hash = line.strip().split(",")
23
24         if stored_username == username:
25             if bcrypt.checkpw(password.encode('utf-8'), stored_hash.encode('utf-8')):
26                 print("Login successful!")
27                 return
28
29     print("User not found!")
30     file.close()
31
32 # Example usage
33 register_user("john_doe", "securepassword123")
34 login_user("john_doe", "securepassword123")
35
36 PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
37
38 PS C:\Myfiles\Attachments\Desktop\Ai-Assist> & C:/Users/sushm/AppData/Local/Programs/Python/Python312/python.exe "c:/Myfiles/Attachments/De
Assist/assgn20.5 task1.py"
User registered successfully!
Login successful!
```

```
17 # Function to verify login
18 def login_user(username, password):
19     file = open("users_secure.txt", "r")
20
21     for line in file:
22         stored_username, stored_hash = line.strip().split(",")
23
24         if stored_username == username:
25             if bcrypt.checkpw(password.encode('utf-8'), stored_hash.encode('utf-8')):
26                 print("Login successful!")
27                 return
28             else:
29                 print("Incorrect password!")
30                 return
31
32     print("User not found!")
33     file.close()
34
35 # Example usage
36 register_user("john_doe", "securepassword123")
37 login_user("john_doe", "securepassword123")
38
39 PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
40
41 PS C:\Myfiles\Attachments\Desktop\Ai-Assist> & C:/Users/sushm/AppData/Local/Programs/Python/Python312/python.exe "c:/Myfiles/Attachments/Desktop/Ai-
Assist/assgn20.5 task1.py"
User registered successfully!
Login successful!
```

Observation:

- Password is hashed using bcrypt
- Original password is not stored
- Uses salt automatically
- Even same password → different hash
- Secure against attacks like:
- brute force (harder)
- rainbow table attacks

Task 2 – Hardcoded Secrets Detection

Evaluate AI-generated code for hardcoded API keys or credentials.

Prompt:

Generate a Python script that connects to an external API (OpenWeather) and retrieves weather data. Then evaluate whether API keys are securely handled and improve the code using environment variables instead of hardcoding.

Code:

```
1 #Evaluate AI-generated code for hardcoded API keys or credentials.
2 #Ask AI to generate a script that connects to an external API.
3 #Identify if API keys/passwords are written directly in code.
4 #Refactor using environment variables or config files.
5 #insecure
6 import requests
7 def get_weather():
8     api_key = "12345ABCDE" # ❌ Hardcoded secret (insecure)
9     url = f"https://api.openweathermap.org/data/2.5/weather?q=Chennai&appid={api_key}"
10
11     response = requests.get(url)
12     print(response.json())
13
14 get_weather()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Myfiles\Attachments\Desktop\Ai-Assist> & C:/Users/sushm/AppData/Local/Programs/Python/Python312/python.exe "c:/Myfiles/Attachments/Desktop/Ai-Assist/assgn20.5 task2.py"
{'cod': 401, 'message': 'Invalid API key. Please see https://openweathermap.org/faq#error401 for more info.'}
```

```
3 #Identify if API keys/passwords are written directly in code.
4 #Refactor using environment variables or config files.
5 #insecure
6 import requests
7 def get_weather():
8     api_key = "12345ABCDE" # ❌ Hardcoded secret (insecure)
9     url = f"https://api.openweathermap.org/data/2.5/weather?q=Chennai&appid={api_key}"
10
11     response = requests.get(url)
12     print(response.json())
13
14 get_weather()
```

```
16 import requests
17
18 def get_weather():
19     api_key = "98537eb2ffca013b91722703fd910690" # ✅ your API key
20
21     url = f"https://api.openweathermap.org/data/2.5/weather?q=Chennai&appid={api_key}"
22
23     response = requests.get(url)
24
25     print("Status Code:", response.status_code)
26     print(response.json())
27
28 get_weather()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Myfiles\Attachments\Desktop\Ai-Assist> & C:/Users/sushm/AppData/Local/Programs/Python/Python312/python.exe "c:/Myfiles/Attachments/Desktop/Ai-Assist/assgn20.5 task2.py"
{'cod': 401, 'message': 'Invalid API key. Please see https://openweathermap.org/faq#error401 for more info.'}
Status Code: 401
{'cod': 401, 'message': 'Invalid API key. Please see https://openweathermap.org/faq#error401 for more info.'}
PS C:\Myfiles\Attachments\Desktop\Ai-Assist>
```

Observation:

The AI-generated code initially stored API keys directly in the source code, which is insecure and can lead to unauthorized access if the code is shared. Even with a valid key, hardcoding remains a risk. This was improved by using environment variables, which keeps sensitive information outside the code and ensures secure handling of credentials.

Task 3 – File Upload Security Validation

Test an AI-generated file upload program for unsafe file handling.

Prompt:

Generate a Flask-based file upload application. Then analyze it for security issues such as unsafe file handling and improve it by adding validation for file type, size, and secure filename handling.

Code:

```

❏ assgn20.5 task3.py > upload_file
1  #Test an AI-generated file upload program for unsafe file handling.
2  #Prompt AI to generate a Flask file upload example.
3  #Check for vulnerabilities like uploading executable files.
4  #Add validation for file type, size, and secure filename handling.
5  """from flask import Flask, request
6
7  app = Flask(__name__)
8
9  @app.route('/upload', methods=['POST'])
10 def upload_file():
11     file = request.files['file']
12     file.save(file.filename) # ❌ Unsafe
13     return "File uploaded!"
14
15 if __name__ == '__main__':
16     app.run(debug=True)"""
17
18
19 from flask import Flask, request
20 from werkzeug.utils import secure_filename
21 import os
22

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

❖ PS C:\Myfiles\Attachments\Desktop\Ai-Assist> & C:/Users/sushm/AppData/Local/Programs/Python/Python312/python.exe "c:/Myfiles/Attachments/Desktop/Ai-Assist/assgn20.5 task3.py"
* Serving Flask app 'assgn20.5 task3'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 784-284-462
127.0.0.1 - - [02/Apr/2026 14:21:13] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [02/Apr/2026 14:21:13] "GET /favicon.ico HTTP/1.1" 404 -
127.0.0.1 - - [02/Apr/2026 14:21:15] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [02/Apr/2026 14:21:25] "POST /upload HTTP/1.1" 200 -

```

```

23 app = Flask(__name__)
24
25 # Upload folder
26 UPLOAD_FOLDER = 'uploads'
27 os.makedirs(UPLOAD_FOLDER, exist_ok=True)
28
29 # Allowed file types
30 ALLOWED_EXTENSIONS = {'png', 'jpg', 'jpeg', 'pdf'}
31
32 # Max file size (2MB)
33 app.config['MAX_CONTENT_LENGTH'] = 2 * 1024 * 1024
34
35
36 # Check file type
37 def allowed_file(filename):
38     return '.' in filename and filename.rsplit('.', 1)[1].lower() in ALLOWED_EXTENSIONS
39
40
41 # Home route (IMPORTANT)
42 @app.route('/')
43 def home():
44     ...

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

❖ PS C:\Myfiles\Attachments\Desktop\Ai-Assist> & C:/Users/sushm/AppData/Local/Programs/Python/Python312/python.exe "c:/Myfiles/Attachments/Desktop/Ai-Assist/assgn20.5 task3.py"
* Serving Flask app 'assgn20.5 task3'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 784-284-462
127.0.0.1 - - [02/Apr/2026 14:21:13] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [02/Apr/2026 14:21:13] "GET /favicon.ico HTTP/1.1" 404 -
127.0.0.1 - - [02/Apr/2026 14:21:15] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [02/Apr/2026 14:21:25] "POST /upload HTTP/1.1" 200 -

```

```

52
53 # Upload route
54 @app.route('/upload', methods=['POST'])
55 def upload_file():
56     if 'file' not in request.files:
57         return "No file part"
58
59     file = request.files['file']
60
61     if file.filename == '':
62         return "No selected file"
63
64     if file and allowed_file(file.filename):
65         filename = secure_filename(file.filename)
66         file.save(os.path.join(UPLOAD_FOLDER, filename))
67         return "File uploaded successfully!"
68
69     return "Invalid file type!"
70
71

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

❖ PS C:\Myfiles\Attachments\Desktop\Ai-Assist> & C:/Users/sushm/AppData/Local/Programs/Python/Python312/python.exe "c:/Myfiles/Attachments/Desktop/Ai-Assist/assgn20.5 task3.py"
* Serving Flask app 'assgn20.5 task3'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 784-284-462
127.0.0.1 - - [02/Apr/2026 14:21:13] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [02/Apr/2026 14:21:13] "GET /favicon.ico HTTP/1.1" 404 -
127.0.0.1 - - [02/Apr/2026 14:21:15] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [02/Apr/2026 14:21:25] "POST /upload HTTP/1.1" 200 -

```

Upload File

17.1 Assignment.pdf

File uploaded successfully!

Observation:

The AI-generated file upload code was initially insecure as it allowed unrestricted file uploads, used unsafe filenames, and lacked validation, making it vulnerable to attacks like uploading malicious files. The improved version adds file type restrictions, size limits, and uses secure filename handling, ensuring safe and controlled file uploads.

Task 4 – Command Injection Prevention

Analyze an AI-generated Python script that executes system commands.

Prompt:

Generate a Python script that executes system commands using user input. Then analyze it for command injection vulnerabilities and improve it using safe subprocess methods and input validation.

Code:


```
◆ assgn20.5 task4.py > ...
1  #Analyze an AI-generated Python script that executes system commands.
2  ##Ask AI to generate code using os.system() or subprocess.
3  #Identify command injection risks with user input.
4  #Fix using safe subprocess calls and input validation.
5  import os
6
7  def run_command():
8      user_input = input("Enter a command: ")
9      os.system(user_input)  # ✗ Dangerous
10
11  run_command()
12
13  import subprocess
14
15  def run_command():
16      filename = input("Enter file name: ")
17
18      if not filename.replace('.', '').isalnum():
19          print("Invalid input!")
20          return
21
22      try:
23          result = subprocess.run(
24              ["dir", filename],
25              capture_output=True,
26              text=True,
27              check=True
28          )
29          print(result.stdout)
30      except subprocess.CalledProcessError:
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Myfiles\Attachments\Desktop\Ai-Assist> & C:/Users/sushm/AppData/Local/Programs/Python/Python312/python.exe "c:/Myfiles/Attachments/Desktop/Ai-Assist/as
◆ PS C:\Myfiles\Attachments\Desktop\Ai-Assist> & C:/Users/sushm/AppData/Local/Programs/Python/Python312/python.exe "c:/Myfiles/Attachments/Desktop/Ai-Assist/as
Enter a command: dir
Volume in drive C is Windows
Volume Serial Number is A4AC-9906

Directory of C:\Myfiles\Attachments\Desktop\Ai-Assist

02-04-2026  14:22    <DIR>          .
25-03-2026  22:10    <DIR>          ..
01-04-2026  09:21             387 1.sql
23-03-2026  14:45           5,130 assgn14.1 task1.html
23-03-2026  14:43           2,169 assgn14.1 task2.html
```

```
◆ assgn20.5 task4.py > ...
15  def run_command():
16      filename = input("Enter file name: ")
17
18      if not filename.replace('.', '').isalnum():
19          print("Invalid input!")
20          return
21
22      try:
23          result = subprocess.run(
24              ["dir", filename],
25              capture_output=True,
26              text=True,
27              check=True
28          )
29          print(result.stdout)
30      except subprocess.CalledProcessError:
31          print("Error executing command")
32
33  run_command()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Myfiles\Attachments\Desktop\Ai-Assist> & C:/Users/sushm/AppData/Local/Programs/Python/Python312/python.exe "c:/Myfiles/Attachments/Desktop/Ai-Assist/as
02-04-2026  14:22    <DIR>          .
25-03-2026  22:10    <DIR>          ..
01-04-2026  09:21             387 1.sql
23-03-2026  14:45           5,130 assgn14.1 task1.html
23-03-2026  14:43           2,169 assgn14.1 task2.html
23-03-2026  14:43           4,577 assgn14.1 task3.html
23-03-2026  14:45           3,367 assgn14.1 task4.html
23-03-2026  14:47           2,559 assgn14.1 task5.html
24-03-2026  11:53             275 assgn15.4 task1.py
25-03-2026  09:17             656 assgn15.4 task2.py
24-03-2026  22:07           1,353 assgn15.4 task3.py
24-03-2026  22:13           1,717 assgn15.4 task4.py
```

```
15 def run_command():
16     try:
17         print("Invalid input!")
18         return
19
20     try:
21         result = subprocess.run(
22             ["dir", filename],
23             capture_output=True,
24             text=True,
25             check=True
26         )
27         print(result.stdout)
28     except subprocess.CalledProcessError:
29         print("Error executing command")
30
31 run_command()
32
33
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Myfiles\Attachments\Desktop\Ai-Assist> & C:/Users/sushm/AppData/Local/Programs/Python/Python312/python.exe "c:/Myfiles/Attachments/Desktop/Ai-Assist/task2.py"
25-03-2026 09:17 656 assgn15.4 task2.py
24-03-2026 22:07 1,353 assgn15.4 task3.py
24-03-2026 22:13 1,717 assgn15.4 task4.py
25-03-2026 09:17 3,149 assgn15.4 task5.py
01-04-2026 09:21 1,229 assgn16.4 task1.sql
01-04-2026 09:21 1,364 assgn16.4 task2.sql
01-04-2026 09:21 1,294 assgn16.4 task3.sql
01-04-2026 09:21 1,230 assgn16.4 task4.sql
01-04-2026 09:21 1,384 assgn16.4 task5.sql
30-03-2026 22:13 763 assgn17.1 task1.py
01-04-2026 09:21 1,004 assgn17.1 task2.py
30-03-2026 22:44 1,356 assgn17.1 task3.py
```

Observation:

The AI-generated code initially used `os.system()` with direct user input, making it vulnerable to command injection attacks where malicious commands can be executed. The improved version replaces it with `subprocess.run()` without using a shell and adds input validation to restrict unsafe characters. This ensures that only safe commands are executed and prevents unauthorized system access.

Task 5 – Secure Input Handling in Web Forms

Check AI-generated web form code for improper input validation.

Prompt:

Generate an HTML and Flask-based feedback form. Then analyze it for improper input validation and unsafe rendering, and improve it using server-side validation and input sanitization.

Code:

```
templates > form.html > form
1  <form method="POST" action="/submit">
2    <h2>Feedback Form</h2>
3
4    Name: <input type="text" name="name" required><br><br>
5
6    Feedback:<br>
7    <textarea name="feedback" required></textarea><br><br>
8
9    <input type="submit" value="Submit">
10 </form>
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Myfiles\Attachments\Desktop\Ai-Assist> python "assgn20.5 task.py"
PS C:\Myfiles\Attachments\Desktop\Ai-Assist> python "assgn20.5 task5.py"
* Serving Flask app 'assgn20.5 task5'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 784-284-462
127.0.0.1 - - [03/Apr/2026 12:21:12] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [03/Apr/2026 12:21:12] "GET /favicon.ico HTTP/1.1" 404 -
127.0.0.1 - - [03/Apr/2026 12:21:23] "POST /submit HTTP/1.1" 200 -
```

```

assgn20.5 task5.py > ...
1  #Check AI-generated web form code for improper input validation.
2  #Ask AI to generate an HTML + Flask feedback form.
3  # Identify missing validation and unsafe rendering.
4  #Fix using server-side validation and sanitization.
5  from flask import Flask, render_template, request
6  from markupsafe import escape
7
8  app = Flask(__name__)
9
10 @app.route('/')
11 def home():
12     return render_template('form.html')
13
14 @app.route('/submit', methods=['POST'])
15 def submit():
16     name = request.form.get('name', '').strip()
17     feedback = request.form.get('feedback', '').strip()
18
19     # Validation
20     if not name.isalnum():
21         return "Invalid name!"
22
23     if len(feedback) > 200:
24         return "Feedback too long!"
25
26     # Sanitization
27     name = escape(name)
28     feedback = escape(feedback)
29
30     return f"Thank you {name}, your feedback: {feedback}"
31
32 if __name__ == '__main__':
33     app.run(debug=True)

```

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\Myfiles\Attachments\Desktop\Ai-Assist> python "assgn20.5 task5.py"
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 784-284-462
127.0.0.1 - - [03/Apr/2026 12:21:12] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [03/Apr/2026 12:21:12] "GET /favicon.ico HTTP/1.1" 404 -
127.0.0.1 - - [03/Apr/2026 12:21:23] "POST /submit HTTP/1.1" 200 -

```

Observation:

The AI-generated form initially lacked proper input validation and directly rendered user input, making it vulnerable to injection attacks like XSS. The improved version adds server-side validation to restrict invalid inputs and uses `escape()` to sanitize user data before displaying it. This ensures that malicious scripts cannot be executed and the application handles user input securely.